

L20-CodeForge: Auditable Test-Time Scaling for Code Generation on a Single GPU

Yin Li
University of Birmingham

May 2026

Abstract

Large code models are often evaluated in settings that assume substantial serving budgets, broad sampling, and opaque benchmark pipelines. This paper studies a narrower question: how much reliable coding performance can be extracted from a 7B-class open model when the compute budget is a single NVIDIA L20 GPU and hidden tests are reserved for measurement. We describe L20-CodeForge, an auditable post-training and inference stack for code generation. The system separates public-side candidate generation, public-test selection, repair, and verifier experiments from private hidden-test replay. On the full 1,055-task LiveCodeBench **release_v6** code-generation suite, Qwen2.5-Coder-7B-Instruct improves from 297 solved tasks under greedy decoding to 403 solved tasks with eight-sample public-test selection, a gain of 10.05 percentage points. On EvalPlus, the clean public-signal system improves HumanEval+ from 84.8% to 92.7% and MBPP+ from 72.2% to 81.7%. We also report negative results: repeated public-feedback repair can overfit visible tests, input-only differential tests do little without multiple public-passing candidates, and an uncalibrated expected-output verifier can regress hidden replay. The main claim is not that a new model checkpoint outperforms the base model; rather, the result is a reproducible case study in benchmark hygiene, test-time scaling, and failure-aware post-training infrastructure under a small-GPU constraint.

1 Introduction

Execution is both the advantage and the trap of code generation. It gives a cheap verifier for many tasks, but it also creates easy ways to overstate progress: select on visible examples, tune against stale hidden tests, or report small optimistic subsets. This tension is sharper for resource-constrained post-training. A single accelerator cannot support the search budgets or training sweeps used by large labs, so every evaluation decision carries more weight.

This paper reports a small-GPU study rather than a new frontier model. We use a single NVIDIA L20 as the target hardware and ask what can be gained by improving the surrounding system: candidate generation, public-test selection, repair, candidate health auditing, hidden replay, and cross-benchmark scorecards. The model in the main experiment is Qwen2.5-Coder-7B-Instruct. Hidden LiveCodeBench tests are used only for final replay and audit, not for candidate selection or repair.

The study makes four contributions.

1. We define an auditable single-GPU evaluation protocol for code generation with a strict public/private boundary.

2. We report full-suite LiveCodeBench `release_v6` and EvalPlus results with committed artifacts, hashes, and re-run commands.
3. We analyze several verifier and repair variants, including negative results that would be hidden by a headline-only report.
4. We release the supporting repository as a reproducibility artifact with CI, scorecards, benchmark summaries, and exact hash checks.

2 Claim Boundary

The central result in this paper is a system result. It should not be read as a claim that the underlying Qwen2.5-Coder-7B-Instruct weights have improved. We make three boundaries explicit.

System-level gains. The LiveCodeBench headline result uses eight sampled candidates and public-test selection. This is a valid solver configuration, but it is different from a single greedy model evaluation. Whenever we compare against greedy decoding, the comparison is between a baseline model call and a bounded public-side inference system.

Public signals only during selection. Public tests, public examples, starter code, and public prompt metadata may be used by the solver. Hidden tests are used only after a candidate has already been selected. No hidden expected outputs, hidden failure messages, or hidden pass/fail labels enter generation, repair, verifier prompts, or tie-breaking.

Negative results remain part of the evidence. Several plausible improvements were rejected because they failed hidden replay or cross-benchmark checks. These failures are reported because they define the boundary between useful test-time scaling and visible-test overfitting.

3 Related Work and Positioning

Code-generation benchmarks. LiveCodeBench was introduced to reduce contamination and broaden code-model evaluation beyond static HumanEval-style tasks, including code generation, self-repair, code execution, and test-output prediction [3]. EvalPlus showed that common code benchmarks can miss incorrect programs because their tests are insufficient; it augments HumanEval and MBPP with many additional tests and can change model rankings [7]. These benchmarks motivate the two evaluation choices used here: full-suite replay on LiveCodeBench and cross-checking with EvalPlus.

Qwen2.5-Coder provides the open 7B base model used in the main experiments [10]. The paper does not compare against larger proprietary systems because the research question is whether a small open model can be used more reliably under a single-GPU budget.

Test-time scaling and execution-guided selection. AlphaCode demonstrated that large-scale sampling followed by program-behavior filtering is central to competitive-programming performance [6]. CodeT generates tests and uses execution agreement to select among samples [1]. LEVER trains an execution-aware verifier to rerank generated programs [9]. S* studies test-time scaling for code and reports large gains from hybrid search and selection on LiveCodeBench [5]. L20-CodeForge follows the same broad line of work but restricts the compute budget and emphasizes auditable artifacts over maximum score.

Verified data and code RL. CodeRL uses unit-test and critic feedback for reinforcement learning and test-time regeneration [4]. ACECoder synthesizes test cases to train reward models and run code RL, reporting gains across HumanEval, MBPP, BigCodeBench, and LiveCodeBench [13]. Recent post-training work also shows that verified competitive-programming data and high-quality tests can substantially improve small models. rStar-Coder constructs a large verified reasoning dataset and reports large LiveCodeBench gains for Qwen2.5 models [8]. HardTests studies high-quality test synthesis for difficult coding problems and finds that stronger tests improve both evaluation and training signal [2]. X-Coder reports a Qwen2.5-derived RLVR model trained with fully synthetic tasks, solutions, and tests [12]. These results suggest that durable model-weight gains need verified data and calibrated rewards, not only prompt changes.

Execution feedback and refinement. OpenCodeInterpreter integrates code generation with execution and refinement through multi-turn feedback [14]. Reflexion uses verbal feedback to improve language-agent behavior across tasks including coding [11]. L20-CodeForge differs in emphasis: the repair loop is not treated as an unconstrained self-improvement process. It is bounded by a public/private evaluation contract, and repeated feedback is reported as a possible overfitting source when hidden replay does not improve.

Line of work	Main idea	Difference in this paper
AlphaCode, CodeT, LEVER, S*	Sampling, execution agreement, verifier or test-time selection	Same family of ideas, but constrained to one L20 and reported with full artifact hashes and negative results
CodeRL, ACECoder	RL or reward modeling from unit tests and synthesized tests	Treated as the next model-weight direction; the present claim remains inference-system level
rStar-Coder, HardTests, X-Coder	Verified synthetic tasks, tests, and reasoning data for stronger code models	Used as evidence that verified data is required for durable single-sample gains
OpenCodeInterpreter, Reflexion	Execution/reflection feedback loops for code or agents	This paper caps public-feedback repair and shows when repeated feedback overfits visible tests
LiveCodeBench, EvalPlus	Stronger contamination and test adequacy standards for code evaluation	Used as measurement infrastructure and as guardrails against optimistic subset results

Table 1: Positioning against the closest research lines.

Taken together, this literature suggests a conservative boundary for the present work. Public-test selection is a legitimate test-time system intervention, but durable model-weight claims require verified training data, calibrated reward signals, and held-out single-sample gains. The experiments below are therefore reported as an auditable inference and post-training infrastructure result, not as evidence that the base checkpoint has improved.

4 Design and Protocol

4.1 Public and Private Signals

Each benchmark problem is treated as two objects. The public object contains the prompt, starter code when available, visible examples, and public tests. It may be used for candidate generation,

public-test selection, and repair. The private object contains hidden tests. It is used only for final replay and audit.

The protocol intentionally permits public-test selection because public tests are available to any solver. It does not permit hidden-test selection, hidden failure feedback, or retuning against hidden outcomes. Negative outcomes are kept in the artifact record because they identify where public-side methods fail to generalize.

4.2 Task Materialization

For LiveCodeBench, the generation host receives a prompt/public-only parquet. That file contains the problem statement, starter code where available, and public examples/tests, but not the private tests. The full `release_v6` JSONL with private tests is materialized separately for hidden replay. Its SHA-256 hash is recorded in the benchmark package; the file itself is not committed because it is large and contains private test payloads.

Each saved generation file is immutable once it becomes a reported artifact. Later selection and audit steps read from saved generations rather than regenerating candidates. This makes the selection protocol inspectable and prevents accidental score drift from sampling randomness.

4.3 Candidate Generation

For the main LiveCodeBench run, the model is Qwen2.5-Coder-7B-Instruct. Greedy decoding uses one completion with temperature 0. The public-selection runs use temperature 0.8, top-p 0.95, a 2048-token completion budget, and either four or eight samples per task. The eight-sample run reuses the first four candidates from the four-sample run and generates only the missing candidates.

Generation is checkpointed by task so interrupted runs can resume without discarding completed samples. This matters on the L20 because a full eight-sample LiveCodeBench generation pass took 28,480.919 seconds.

4.4 Public-Test Selection

For each task, candidates are executed against public tests only. If at least one candidate passes all public tests, the selector chooses the shortest public-passing program. If none pass all public tests, it chooses the shortest candidate among those with the best public score. The shortest-solution tie-breaker is deliberately simple: it avoids hidden labels, avoids learned reranking that would need extra calibration, and makes the selection rule easy to audit.

The selector also records public-pass counts, selected indices, and selected public scores. For the eight-sample LiveCodeBench run, 618 of 1,055 selected candidates passed all public tests. Hidden replay later showed 403 hidden passes, leaving a large public-pass/hidden-fail gap that motivates repair and verifier work.

4.5 Repair and Verifier Experiments

Repair is treated as a public-side second pass. The repair prompt may include the original problem, public examples, the candidate code, and public-test failure messages. It may not include private failures. We evaluated three repair variants on the X-Coder medium control slice: starter-prefix prompting, public-only second-pass repair, and public-feedback repair.

Verifier experiments are evaluated with the same rule: they may only consume public-side signals before selection. Candidate-aware behavior tests and expected-output verifier prompts were

therefore built from generated or public inputs, not private labels. A verifier variant is allowed into a headline result only if it improves hidden replay without failing the generalization scorecard.

4.6 Hidden Replay and Scorecards

After public-side selection, the selected candidate is replayed against private tests using the LiveCodeBench evaluator. Hidden replay is chunked so a single-problem worker failure does not invalidate the full run. In the reported eight-sample run, one worker failure occurred on question id 3637; it is counted as a hidden failure rather than omitted.

The generalization scorecard combines full LiveCodeBench with EvalPlus HumanEval+ and MBPP+. Any future selector, repair loop, adapter, or verifier must improve the target benchmark without failing this scorecard.

4.7 Hardware and Model

The target hardware is a single NVIDIA L20 GPU. The main LiveCodeBench and EvalPlus experiments use Qwen2.5-Coder-7B-Instruct without an adapter. The reported LiveCodeBench system result uses temperature 0.8, top-p 0.95, eight samples per task, public-test selection, and hidden-test replay.

5 System Modules

Figure 1 summarizes the implementation. Candidate generation and repair are public-side operations. Hidden replay is outside the selection loop. The scorecard combines LiveCodeBench and EvalPlus to catch benchmark-specific overfitting.

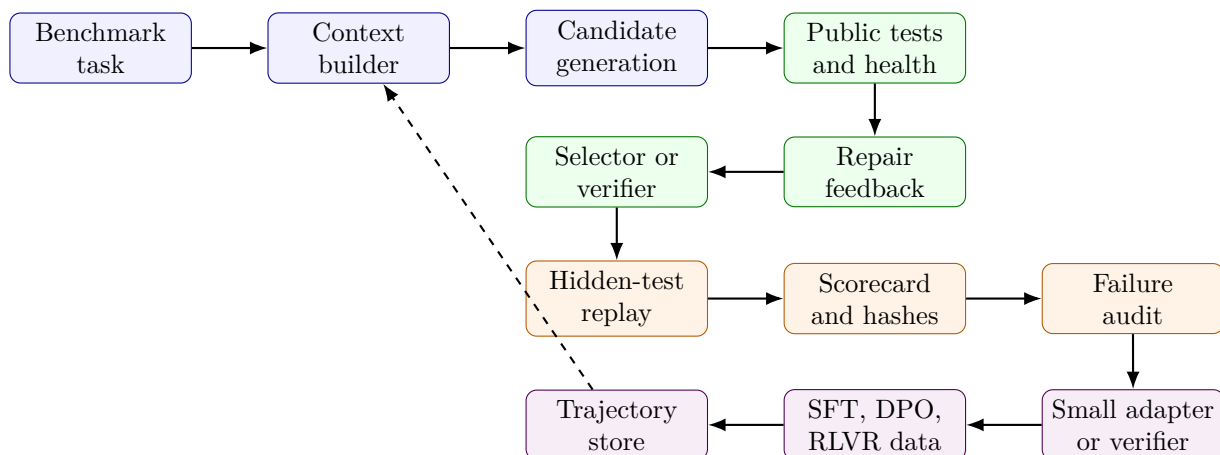


Figure 1: L20-CodeForge evaluation loop. Public signals drive generation, selection, and repair. Private tests are reserved for final replay and audits.

The system is organized so each stage can be rerun without changing the contract of the adjacent stages. The context builder emits public-only prompt records. The generator emits immutable candidate files. The selector consumes candidate files and public execution outcomes, but never private labels. Hidden replay consumes exactly one selected candidate per task. The scorecard then joins benchmark summaries, hashes, and failure counts into a read-only report. This separation is

a practical guardrail: a new repair policy, verifier, or adapter can be swapped into the public side while preserving the same hidden replay and scorecard path.

6 Experiments

6.1 LiveCodeBench

Table 2 reports the main full-suite LiveCodeBench result. The greedy baseline solves 297 of 1,055 tasks. Four-sample public-test selection solves 378 tasks. Eight-sample selection solves 403 tasks, an absolute gain of 106 tasks over greedy decoding.

Model	Protocol	Samples	Solved	Total	Pass@1
Qwen2.5-Coder-7B-Instruct	greedy	1	297	1055	28.15
Qwen2.5-Coder-7B-Instruct	public selection	4	378	1055	35.83
Qwen2.5-Coder-7B-Instruct	public selection	8	403	1055	38.20

Table 2: LiveCodeBench **release_v6** full-suite results. Public selection uses only visible public tests; hidden tests are used for final replay.

The relative improvement from greedy decoding to eight-sample public selection is 35.69%. The result should be interpreted as bounded test-time scaling under a public-test solver, not as a new model checkpoint.

The gain is not uniform across difficulty levels. Easy tasks improve from 206 to 260 solved tasks, medium tasks from 82 to 124, and hard tasks from 9 to 19. The hard-task result remains weak in absolute terms, which is consistent with the need for stronger algorithmic candidates rather than only better selection.

Slice	Greedy solved	System solved	Total	Greedy	System
Easy	206	260	322	63.98	80.75
Medium	82	124	383	21.41	32.38
Hard	9	19	350	2.57	5.43

Table 3: LiveCodeBench difficulty breakdown. Values in the last two columns are percentages.

Platform	Solved	Total	Pass@1
AtCoder	199	602	33.06
Codeforces	5	9	55.56
LeetCode	199	444	44.82

Table 4: LiveCodeBench platform breakdown for the eight-sample public-selection run. Values in the last column are percentages.

Failure class	Count
Wrong answer	438
Time limit exceeded	147
Runtime error	66
Fatal chunk failure counted as failure	1

Table 5: Failure classes after eight-sample public selection and hidden replay. Wrong answers dominate, which points to algorithmic candidate quality rather than only formatting or endpoint repair.

6.2 EvalPlus

Table 6 reports the EvalPlus guardrail. The clean system rows use public prompt and base-test signals, but not EvalPlus extra tests, for selection. Both HumanEval+ and MBPP+ improve over greedy decoding.

Dataset	Greedy base	System base	Greedy plus	System plus
HumanEval	89.0	98.2	84.8	92.7
MBPP	82.8	96.0	72.2	81.7

Table 6: EvalPlus results. Values are pass@1 percentages. The plus columns use the stricter EvalPlus tests for scoring, not for candidate selection.

6.3 Artifacts, Hashes, and Runtime

Table 7 records the main reproducibility artifacts. Long hashes are included because they are the practical link between the paper claim and the repository state.

Artifact	SHA-256
Full LiveCodeBench JSONL	4519422b52d7dd243358ee721ecfe94e26ab364a7af2938c8e5d170a17bcadcd
Eight-sample generations	0950cbbbfcbcd6d716b2c9b556e7e9aa3626211381a95e4c6e4a367bdd95df57
Eight-sample summary	2a0ff919aa15eb9ecdf74824f7bf790a23f6d0197ef74970b6190c60e0e00772
Generalization scorecard	1eb0402378ea25732225b29d7ba367b6111ab3351e54cc7c01fa7646a7a12712
EvalPlus summary	08732bbb76450f92ef3c02fa97a163aba01f71028365072c205c5a3af45d5550

Table 7: Artifact hashes for the reported results.

Operation	Wall time / setting
Eight-sample generation	28,480.919 seconds
Chunked hidden replay	449.914 seconds
Chunk wall-time limit	60 seconds
GPU budget	single NVIDIA L20

Table 8: Runtime and hardware summary for the main LiveCodeBench result.

6.4 X-Coder Control Slice

We also probed IIGroup/X-Coder-RL-Qwen2.5-7B on a medium-difficulty control slice. Automatic and strict starter-prefix checks both scored 0/12. Public-only second-pass repair reached 2/12. A single public-feedback repair round reached 4/12. A second public-feedback round produced visible-test signal but no hidden passes. We therefore treat the first feedback round as useful but not sufficient for broad claims.

Control experiment	Public oracle	Hidden passes	Interpretation
Automatic starter prefix	0/12	0/12	No transfer from targeted first-12 probes
Strict starter prefix	0/12	0/12	Output control did not solve algorithms
Public-only repair	2/12	2/12	First non-targeted positive signal
Fail10 multi-source repair	0/10	0/10	More code-only samples did not help
Public-feedback repair	2/10	2/10	Feedback changed behavior on two tasks
Second feedback round	1/8	0/8	Visible-test overfit signal

Table 9: X-Coder medium control-slice repair and feedback ablations. The combined control-slice status after public-only repair plus one feedback round is 4/12.

7 Failure Analysis

The main failure class on the eight-sample LiveCodeBench run is wrong answer, not syntax or missing entrypoints. This suggests that future work should prioritize better algorithmic candidates and stronger verifier calibration over format-only prompting.

Several attempted improvements did not hold up under hidden replay. Repeated public-feedback repair overfit visible tests on the second round. Input-only differential fuzzing improved candidate-pair coverage on some targets but did not change selection when the pool contained too few public-passing alternatives. An expected-output verifier over generated inputs regressed the targeted replay, indicating that verifier confidence needs calibration before it can safely alter headline selections.

The same pattern appears in the X-Coder control slice. Public-feedback repair is useful for one round, but a second round created a public-passing candidate without a corresponding hidden pass. This is the failure mode the protocol is designed to expose: visible tests can steer the model toward local fixes that do not capture the underlying algorithm.

8 Reproducibility

The repository includes committed benchmark summaries, saved generations, reports, hash manifests, and CI checks. The full hidden-test LiveCodeBench JSONL is not committed. Instead, the repository records materialization commands, source hashes, generation hashes, evaluator outputs, and compact summaries.

For the main LiveCodeBench result, the committed summary hash is:

2a0ff919aa15eb9ecdf74824f7bf790a23f6d0197ef74970b6190c60e0e00772

The generalization scorecard hash is:

1eb0402378ea25732225b29d7ba367b6111ab3351e54cc7c01fa7646a7a12712

The repository also includes a separate reproducibility document with expected command outputs and artifact-hash checks.

The arXiv source package is intentionally small: `main.tex` and `references.bib` are sufficient to rebuild the paper. Generated PDFs, logs, and auxiliary files are excluded from version control and from the source archive.

9 Limitations

The strongest limitation is that the main gain is a system-level gain, not a model-weight gain. The eight-sample selector is useful in practice but should not be confused with a greedy checkpoint improvement. The experiments also use one main base model and one main GPU type. Public tests are legitimate solver signals, but repeated repair on public failures can overfit them. Finally, private benchmark tests cannot be committed directly, so full hidden replay requires local materialization of the benchmark payload.

The paper is also limited by scope. It does not yet include a trained adapter that improves greedy LiveCodeBench performance, a calibrated learned verifier, or a verified synthetic-data pipeline comparable to rStar-Coder or X-Coder. The appropriate next claim would require held-out single-sample gains, not only stronger public-side selection.

10 Conclusion

L20-CodeForge shows that a careful single-GPU system can extract a meaningful amount of additional coding performance from a 7B open model while preserving a clean benchmark boundary. The useful parts are not only the positive results, but also the audit trail: public selection helps, first-round repair can help, and several plausible verifier variants fail without calibration. The next step is to convert this infrastructure into a verified-data and RLVR pipeline that improves single-sample model behavior on held-out tasks.

References

- [1] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. CodeT: Code generation with generated tests, 2022.
- [2] Zhongmou He, Yee Man Choi, Kexun Zhang, Jiabao Ji, Junting Zhou, Dejia Xu, Ivan Bercovich, Aidan Zhang, and Lei Li. HardTests: Synthesizing high-quality test cases for LLM coding, 2025.
- [3] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code, 2024.
- [4] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning, 2022.

- [5] Dacheng Li, Shiyi Cao, Chengkun Cao, Xiuyu Li, Shangyin Tan, Kurt Keutzer, Jiarong Xing, Joseph E. Gonzalez, and Ion Stoica. S*: Test time scaling for code generation, 2025.
- [6] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Remi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.
- [7] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation, 2023.
- [8] Yifei Liu, Li Lyna Zhang, Yi Zhu, Bingcheng Dong, Xudong Zhou, Ning Shang, Fan Yang, and Mao Yang. rStar-Coder: Scaling competitive code reasoning with a large-scale verified dataset, 2025.
- [9] Ansong Ni, Srini Iyer, Dragomir Radev, Ves Stoyanov, Wen-tau Yih, Sida I. Wang, and Xi Victoria Lin. LEVER: Learning to verify language-to-code generation with execution, 2023.
- [10] Qwen Team. Qwen2.5-Coder technical report. <https://qwenlm.github.io/blog/qwen2.5-coder-family/>, 2024.
- [11] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023.
- [12] Jie Wu, Haoling Li, Xin Zhang, Jiani Guo, Jane Luo, Steven Liu, Yangyu Huang, Ruihang Chu, Scarlett Li, and Yujiu Yang. X-Coder: Advancing competitive programming with fully synthetic tasks, solutions, and tests, 2026.
- [13] Huaye Zeng, Dongfu Jiang, Haozhe Wang, Ping Nie, Xiaotong Chen, and Wenhui Chen. ACE-CODER: Acing coder RL via automated test-case synthesis, 2025.
- [14] Tianyu Zheng, Ge Zhang, Tianhao Shen, Xueling Liu, Bill Yuchen Lin, Jie Fu, Wenhui Chen, and Xiang Yue. OpenCodeInterpreter: Integrating code generation with execution and refinement, 2024.